

TEL102 - Práctico 4

Semestre 2, 2025

Fecha: 17/11/2025, **Duración:** 1h 10min (30 min Parte A, 40 min Parte B)

Instrucciones generales

- **Parte A:** trabajo en papel, sin computador. Se analiza y diseña una clase Qt en C++ (sin interfaz), usando **signals y slots**.
- **Parte B:** implementación en Qt reutilizando lo diseñado en la Parte A.

Se permite usar **IA** en la Parte B, pero debe registrarse en [Readme.md](#):

- Datos del estudiante.
- Prompts usados.
- Modelo y versión.
- Correcciones hechas al código sugerido por la IA.
- Breve reflexión de uso.

Se entregan archivos base del proyecto en Aula USM. Solo se pueden usar elementos **vistos en clases**.

Contexto: TemperatureLogger

Se desea construir una aplicación Qt que permita:

1. Ajustar la temperatura en **Celsius** y **Fahrenheit** mediante diales, manteniendo ambos valores sincronizados.
2. Registrar las **10 últimas temperaturas en °C** y mostrarlas en un **gráfico de línea**.

El programa se organiza en tres componentes:

- **TempConverter**: ya incluida. Convierte temperaturas y emite señales cuando cambian.
- **TempHistory**: clase a completar. Almacena las últimas 10 temperaturas en un `std::vector<float>` y emite señales para actualizar el gráfico.
- **MainWindow**: interfaz principal que integra diales, displays y gráfico, conectando todo mediante signals/slots.

Parte A: Qt sin interfaz + diseño de la aplicación (en papel, sin computador)

En esta parte se trabaja **solo en papel**. No se permite usar computador.

A.1 Clase `TempHistory`

A partir del **esqueleto mínimo** de la clase, usted deberá **completar la clase de forma manual**, agregando todo lo necesario para cumplir los requerimientos.

Usted debe **completar el diseño** agregando:

1. Métodos (obligatorios)

- Agregar temperatura (solo se guardan las últimas 10).
- Obtener cantidad de temperaturas.
- Acceder a una temperatura por índice (`valueAt(i)`).

2. Signals

Por ejemplo:

- Avisar que el historial cambió.
- Avisar que se agregó una nueva temperatura.

3. Slots

- Slot que recibe temperaturas en °C desde `TempConverter`.

Ejemplo de conexión que su slot debe permitir:

```
QObject::connect(&converter,
                &TempConverter::tempCelsiusChanged,
                &history,
                /* su slot */);
```

temphistory.h

```
#ifndef TEMPHISTORY_H
#define TEMPHISTORY_H

#include <QObject>
#include <vector>

// Historial de temperaturas en grados Celsius

class TempHistory : public QObject {
    Q_OBJECT

public:
    TempHistory(QObject *parent = 0);

    // COMPLETAR

private:
    std::vector<float> m_values; // historial de temperaturas en grados
    Celsius
};

#endif // TEMPHISTORY_H
```

Como parte del funcionamiento de la aplicación, **TempConverter** y **TempHistory** se utilizarán en conjunto, estableciendo una conexión para que cada cambio de temperatura en Celsius sea recibido por **TempHistory**. Esta conexión se realiza de la siguiente forma, y requiere que usted defina el slot adecuado dentro de **TempHistory**:

```
TempConverter converter;
TempHistory history;

// Conexión entre TempConverter y TempHistory:
// Cada vez que cambie la temperatura en Celsius, TempHistory debe recibir
ese valor
// mediante un slot definido por usted (por ejemplo: onNewCelsius(int)).
QObject::connect(&converter,
                 &TempConverter::tempCelsiusChanged,
                 &history,
                 /* su slot */);
```

Parte B: Implementación de TemperatureLogger en Qt (en computador)

En esta parte deberá implementar la aplicación **TemperatureLogger** utilizando C++ y Qt, **respetando el diseño realizado en la Parte A**.

Archivos entregados

En AULA USM se entregará un proyecto Qt **incompleto** que incluye:

- **mainwindow.ui** **listo y completo** (interfaz gráfica preconstruida).
 - Archivos **.h** y **.cpp** de **TempConverter** **completos**.
 - Archivos **temphistory.h** y **temphistory.cpp** **incompletos**, que deberá completar según su diseño de la Parte A.
 - Archivos **mainwindow.h** y **mainwindow.cpp** **incompletos**, que deberá completar según su diseño de la Parte A, cumpliendo con los requisitos previos.
 - Archivo de proyecto (**CMakeLists.txt**).
-

B.1 Implementación de TempHistory

En **temphistory.h/.cpp** debe implementar la clase siguiendo **su propio diseño** escrito en la Parte A, incluyendo:

- Métodos para agregar, contar y acceder a temperaturas.
- Signals definidos por usted para notificar cambios.
- Slots definidos por usted para recibir temperaturas desde **TempConverter**.

Para recibir el puntaje completo, la implementación debe coincidir con lo que escribió en papel.

B.2 Integración con la interfaz

El estudiante deberá completar lo siguiente:

- Agregar **las conexiones faltantes**, especialmente aquellas que enlacen:
 - **TempConverter** → **TempHistory** (según el diseño definido en la Parte A)
 - **TempHistory** → **MainWindow** (para actualizar el gráfico)
- Implementar el slot **updateChart()** que:
 - Limpie la serie del gráfico.
 - Recorra el historial utilizando únicamente los métodos públicos de **TempHistory**.
 - Agregue cada punto (**índice, valor**) a la serie.

Solo **TempHistory** debe almacenar y proveer los datos para el gráfico.

Para recibir el puntaje completo, la implementación debe coincidir con lo que escribió en papel.

Forma de entrega

Suba a AULA un archivo comprimido (`.zip`, `.rar`, `.tar.gz`, etc.) que incluya:

- Código fuente (`.h`, `.cpp`, `.ui`).
- Archivo de proyecto `CMakeLists.txt`.
- `Readme.md`.

El proyecto debe compilar y ejecutarse correctamente.

Rúbrica Práctico 4 (10 pts)

Parte A (4 pts)

- **(2 pts)** Diseño de `TempHistory`: métodos correctos, uso de `std::vector<float>`, historial en °C.
- **(1 pt)** Signals y slots coherentes con lo pedido.
- **(1 pt)** Conexiones correctamente diseñadas.

Parte B (6 pts)

- **(3 pts)** Implementación de `TempHistory` consistente con Parte A (0pts si la entrega de parte A es insuficiente para evaluar).
- **(1 pts)** Conexiones faltantes en `MainWindow` (Converter → History, History → Chart).
- **(1 pt)** `updateChart()` funcional usando solo `TempHistory`.
- **(1 pt)** Proyecto compila y funciona correctamente.